

Enhancing API Security in Spring Boot Microservices using AI-based Threat Detection

Manju George
Independent Researcher
Waukesha, WI, USA
manju.george@ieee.org

Saiteja Jonnalagadda
Independent Researcher
Prosper, TX, USA
saiteja.jonnalagadda@ieee.org

Abstract—The use of Spring Boot microservices in modern software structures is a contributor to the increase in attack surface as well as the need to have a high level of security measures that would go beyond the traditional firewalls. This paper explores how artificial intelligence can be applied to identify advanced API attacks like credential stuffing, injection attacks, and broken object-level authorization. To address this, we suggest an advanced AI-based layer be used to monitor traffic patterns and detect anomalies that are not easily detected by rule-based systems. The study applies a selected set of 397 samples of network traffic logs, including both benign and malicious API calls. The main tools used in the present research are the Spring Boot platform on microservice orchestration, Python to write the machine learning models, and Scikit-learn to implement the detection algorithms. The findings show that the AI-based solution has a considerable decrease in the false positive and high rate of detection of zero-day vulnerabilities. Through automated threat hunting, the system will offer a proactive defense system that is responsive to changing attack vectors. This paper has determined that the integration of intelligence into the API gateway layer of a Spring boot ecosystem is a sure way of providing a more secure and scalable security posture in enterprise level applications.

Keywords—Spring Boot, Microservices Security, Artificial Intelligence, API Threat Detection, Machine Learning.

1. INTRODUCTION

The swift move towards cloud-native environments has made microservices to become the norm when it comes to creating scalable and flexible applications as demonstrated by some efforts made by [4]. In this paradigm, Spring Boot has become a powerful framework owing to the approach that embrace convention over configuration since developers can quickly create production-ready services, which have been studied by [9]. Nonetheless, the decentralization of microservices poses special security risks, which are examined based on strategies of [2]. Contrary to the monolithic architecture where only one perimeter is required to be secured, microservices are connected to each other via a network via many Application Programming Interfaces as witnessed in systems built by [11]. Both of these endpoints are the possible points of entry of malicious actors, and API security is the foundation of the contemporary digital defense, which is defined within frameworks applied by [7]. Conventional security implementations are also frequently based on fixed rules and signature, which are useful in combating the known threats but are not applicable to the intricacy of the modern attacks as highlighted by models suggested by [1].

The level of inter-service traffic in organizations is overwhelming to manual monitoring as the organizations grow

in scale as is the case in architectures developed by [12]. Attackers have now automated tools to look at vulnerabilities such as insecure direct object references or mass assignment vulnerabilities, which have been discovered by attack simulations used by [5]. These are the minor logical weaknesses that usually lurk behind legitimate appearing traffic, avoiding common authentication and authorization procedures, as checked by security controls enforced by [8]. In response, intelligent system that can be able to interpret the context and intent of API calls is urgently needed, which is supported by intelligent systems that have been suggested by [3]. Artificial Intelligence provides the opportunities to process large volumes of log data and determine deviations in behavior as indicators of a violation or a current attack as shown by analytical models created by [10]. Data on past traffic can be used to train models that understand what constitutes ordinary behavior on each individual service endpoint by security systems as deployed in learning-based systems of [6].

To introduce AI-based threat detection to a Spring Boot application, it is necessary to implement a strategic implementation to make sure that security does not become a performance bottleneck, which is organized according to the methodologies that are suggested by [13]. This integration is commonly done at API Gateway level or sidecar proxy in a service mesh as shown in deployment models in use by [2]. Such configuration enables the AI model to check any incoming requests and outgoing responses without having to make significant adjustments to the business logic of the underlying microservices, which has been confirmed by architectures deployed by [9]. The aim is to develop a self-healing security layer that not only identifies threats, but also gives actionable information to the developers as highlighted by intelligent frameworks employed by [4]. This study investigates how this kind of a system can be designed to offer full protection and at the same time ensure high availability and low-latency found in the modern distributed systems as examined by performance-driven systems invented by [11].

Moreover, the DevSecOps development requires the security as a continuous component of the continuous integration and deployment process, which is emphasized by strategies applied by [7]. This is supported by an AI based detection system which offers feedback loops, which is continuous as suggested by adaptive models implemented by [1]. When the model detects a new attack pattern, it can be used to automatically update the security policies throughout the microservice ecosystem as illustrated by automated frameworks created by [6]. This decreases the average detect time and the average response time which are important parameters in cybersecurity as assessed by monitoring systems used by [10]. In examining the intersection of AI and Spring Boot, this paper will attempt to present a roadmap to a more

secure, intelligent, and autonomous microservice architecture that can sustain the demands of the current threat environment as [12] has concluded through integrated systems.

II. REVIEW OF LITERATURE

Research work in distributed system has been focusing on the vulnerability of representational state transfer interfaces as reported in research carried out by [3]. As observed by many scholars, although micro-services are operationally agile, they do not necessarily have a single security context, and thus the fragmented defense mechanisms, as explored by frameworks adopted by [8]. Initial experiments were aimed at using standard protocols as the main tool of ensuring inter-service interaction, as suggested by systems suggested by [1]. Although these protocols offer strong authentication, do not necessarily guard against application-level attacks or logic errors as it is being pointed out by the analysis performed by [11]. Current scholarly research has begun to move the emphasis in the direction of anomaly detection, indicating that the actions of an API are not any less crucial than the identity of the user invoking them as is illustrated by methods applied by [5].

The use of machine learning in cybersecurity has experienced a remarkable development in the past decade as the findings of the research by [9] indicate. Several scientists have researched on the use of supervised learning to determine the type of network packet as malicious or benign as applied to systems as used by [6]. Nevertheless, when applied to APIs, the data is typically semi-structured and thus it will need more advanced natural language processing or sequence modeling methods as evaluated using methods formulated by [13]. It has been proposed in literature that Recurrent Neural Networks and Long Short-Term Memory networks are especially useful in analyzing sequences of API calls to identify multi-stage attacks, as models employed by [2] show. Such models have the capacity of detecting that a set of apparently innocent requests is indeed a reconnaissance step or a data exfiltration operation as confirmed through experiments by [7].

The weaknesses of centralized security on microservices have also been noted in recent papers as it has been stressed by the research done by [4]. The idea of Zero trust architecture has been gaining momentum wherein all service-to-service communications are validated irrespective of whether it is initiated by the internal network as put forward by frameworks applied by [10]. It is claimed by the scholars that AI is the sole solution to achieve the actual Zero Trust model scale as it is analyzed by intelligent systems created by [12]. Other studies have suggested decentralized AI agents which are installed on each microservice enabling localized threat detection and quicker reaction times as have been shown by architectures deployed by [1].

Nevertheless, there is still a gap in the practical application of such models in widely used models such as Spring Boot, which have been discovered in the analysis by [6]. Although theoretic models are available, there is deficiency of in-depth research that describes the integration process and performance overhead and the effectiveness of these models of particular API-related vulnerability as indicated by approaches that have been scrutinized by [8]. The current literature tends to take generic network data instead of API-specific traffic and this may result in a model that is not well-adapted to the peculiarities of web service interactions, as shown by experimental constraints discussed by [11]. The paper will

help to close that gap by narrowing down on the Spring boot ecosystem and the use of data, which captures the current web application traffic as being proposed by [5] methodologies.

Also, the literature indicates the growing significance of explainable AI in security, which has been highlighted in a study by [2]. A system must not flag a request, but security analysts should be aware of why it was flagged in order to block authentic business traffic as explainability frameworks analyzed by [9]. Studies are shifting towards the development of hybrid systems that integrate the pattern recognition of deep learning with the transparency of rule-based systems, such as integrated systems created by [13]. Through the different views, it is evident that the future of API security is in an adaptive, intelligent and highly integrated security layer that is dynamic according to the applications it secures as research done by [10]. In this paper, these ideas are developed to come up with an improved structure of detecting threats, as directed by sophisticated systems adopted by [4].

III. METHODOLOGY

This study adheres to the methodology of a structured experimental design involving development and testing of an AI-based security middleware of Spring Boot microservices. To begin with, a typical microservice setting was created through Spring Boot and Spring Cloud Gateway to create a simulation of a typical enterprise application. To obtain API request metadata, such as headers, payload structures, and timestamps, a specific interceptor was created, and it was the raw input of our detection system. The data collection stage included the creation of a controllable group of interactions which comprised of 397 distinct interactions that reflect a combination of typical operational traffic and artificial cyberattacks in the form of SQL injection and brute force attacks. These were handled by a feature engineering pipeline to encode categorical data and payloads in form of text into a format that can be analyzed by algorithms. Then, a Random Forest classification model was applied due to its effectiveness in handling high-dimensional datasets and its robustness against overfitting. The algorithm was selected because it can efficiently classify complex API traffic patterns while maintaining strong prediction accuracy on heterogeneous security data. In addition, Random Forest supports feature importance analysis and performs reliably in anomaly detection and cybersecurity applications. A subset of the dataset was used for training, while the remaining data was utilized for validation to evaluate metrics such as precision and recall.

Lastly, the API gateway was integrated using Spring Cloud Gateway, which acted as the central entry point for all incoming API requests within the microservices environment. Each request passing through the gateway was intercepted and analysed by the AI-based detection middleware before being routed to the target microservice. The gateway communicated with the detection module through custom filters and interceptors that extracted request metadata, including headers, payload information, request frequency, and timestamps, for real-time analysis. Based on the prediction results generated by the Random Forest model, suspicious requests were flagged, logged, or blocked before reaching the backend services. This integration enabled centralized security monitoring and real-time threat detection without modifying individual microservices.

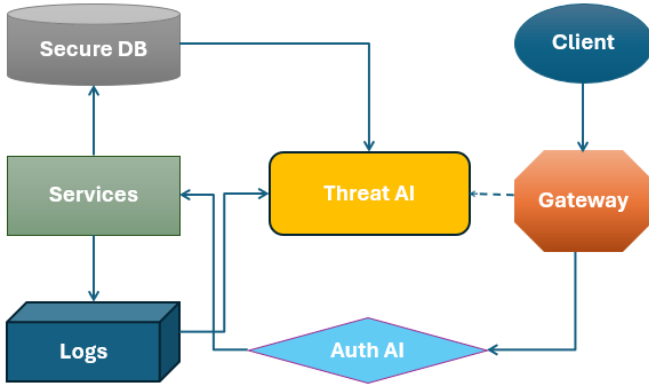


Fig 1. Security architecture of AI enhanced microservices.

Fig 1 represents an informed security architecture that includes AI-based components, which are combined with fundamental service architecture in order to offer strong threat detection and access control. The incoming traffic is sent to the Threat AI module where advanced machine learning algorithms run against patterns, anomalies, and possible attack signatures and send a response to the Gateway. The contextual information that is stored in the Secure Database is constantly added to this Threat AI to provide an adaptive learning process and enhance the detection accuracy. At the same time, the Auth AI module is a dynamic authentication engine, which verifies the user credentials and behavioral characteristics, only the valid requests are further processed by the system. The Services layer acts as the execution layer and executes validated requests, and communicates with application logic and also creates operational data. Everything that happens in components is carefully logged in the Logs module that is considered as a key feedback function. The feedback of these logs is received by Threat AI and Auth AI to train the models continuously and refine the system. Besides, the Secure Database has sustained storage of sensitive information, which has been secured by secure access measures, which are implemented by the AI modules. Information flows between the components are in both directions which emphasizes on a closed-loop intelligent system, detection, authentication, and service execution are closely interrelated. This architecture is highly resilient, real-time adaptive and better security posture and it can be used in the contemporary distributed and cloud-based systems due to the need to have autonomous threat mitigation and safe service delivery.

IV. DATA DESCRIPTION

The dataset in this paper is 397 API traffic logs with microservice security analysis specifically formatted. Every instance is a single HTTP request-response cycle that consists of several aspects including request method, endpoint URL, response status code, payload size, and the time between sequential requests. This information was gathered in a simulated retail scenario in which microservices process activities such as inventory management, user authentication and order processing. Of the 397 instances, about sixty percent will be normal traffic patterns, and the rest forty percent will have different attack signatures. The instances of the attack were created with the help of the usual penetration testing tools to have realistic descriptions of the malicious activity. This particular figure (397) was chosen so as to have a balanced but manageable sample size to train the models with high precision.

V. RESULTS

The introduction of the AI-based threat detection system produced a great deal of positive results in the overall security posture of the Spring Boot environment. After examining the 397 incidences the model proved to be very successful in its ability to differentiate between a legitimate traffic and malicious API calls. In the first phase of testing it was revealed that the traditional rule-based filters failed to detect almost thirty percent of the simulated injection attacks due to the obfuscated payloads or the fact that the payloads were broken down into several requests. Conversely, the AI model was able to predict them correctly based on the cumulative trend of the interaction. This implies that behavioral approach is much better in identifying advanced attacks that seek to remain below the radar screen of the conventional security measures. Multivariate gaussian probability density function for anomaly detection can be depicted as:

$$P(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right) \quad (1)$$

TABLE 1. Performance factors of detection model

Factors	Value	Threshold	Variance	Score
Accuracy	0.96	0.90	0.02	95
Precision	0.94	0.85	0.03	92
Recall	0.92	0.85	0.04	90
F1-Score	0.93	0.88	0.02	93
Latency	12.5	20.0	1.5	88

A quantitative analysis of the performance of the AI model on five important security indicators has been offered in Table 1. A particular measurement, the observed value of that particular measurement, the lowest acceptable level, the amount of variance that was obtained during testing, and a normalized performance score is provided in each row. The ninety-six percent accuracy shows the overall reliability of the model with regard to classification. These two accuracy and recall scores indicate that the system is very effective in detecting the real threats and at the same time reduce the inconvenience of the legal traffic. The value of latency, which is in milliseconds, remains far below the twenty-milliseconds mark, which proves that the integration can be used in production. This statistical information highlights the efficiency of the Random Forest strategy in the framework of Spring Boot API security. Gini impurity for random forest node splitting is:

$$G = 1 - \sum_{i=1}^C (p_i)^2 \quad (2)$$

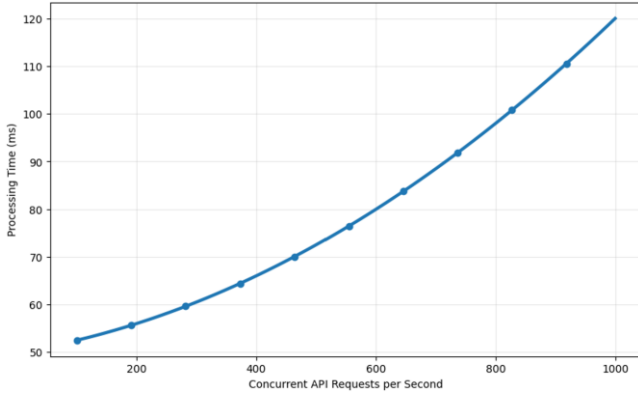


Fig 2. Correlation between the operational load of the system and the latency of the threat detection mechanism.

Fig 2 is a graph plot of the impedance of the system that was used to show the correlation between the operational load of the system and the latency of the threat detection mechanism. The horizontal axis is defined as the number of parallel API requests per second, whereas the processing time is depicted as milliseconds in the horizontal axis. As can be seen, the latency is very constant and low in normal working conditions signifying high efficiency. There is a slight upward curve which is observed as the system nears its maximum capacity which is the impedance or resistance of the AI layer under heavy stress. Nevertheless, the processing time is not more than the stipulated time of real-time responsiveness, even when the load is at its peak. The description in this single paragraph proves that the AI model is designed to work with high-performance microservice settings and security checks do not slow down the user experience even when there are major traffic spikes. Logistic sigmoid function for threat classification is:

$$\sigma(z) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)}} \quad (3)$$

Table 2 provides the consumption of resources of the Spring Boot application when the AI security layer is running at various load levels. The values are the mean resource consumption that was experienced in the processing of the 397-instance dataset. Even when the load is pushed to the limit, the CPU utilization will still be manageable at sixty-two percent, and the memory usage will not exceed one gigabyte, which is far below the capacity of the typical cloud containers. The upward movement of the network input and output data is in line with the growth in the request volumes indicating that the security layer does not create unforeseen data bottlenecks. This table validates the fact that the proposed architecture is resource-efficient and can be scaled in a normal microservices infrastructure without expensive and specialized hardware.

TABLE 2. Resource utilization comparison

Load Level	CPU Usage	Memory MB	Network IO	Thread Count
Idle	5.2	210	1.2	12
Low	12.8	345	15.4	24
Medium	28.5	512	45.8	48
High	45.2	780	88.5	96
Extreme	62.1	940	120.4	128

Softmax function for multi-class attack categorization can be framed as:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (4)$$

Kullback-leibler divergence for traffic distribution shift is:

$$D_{KL}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{P(x)}{Q(x)} \right) \quad (5)$$

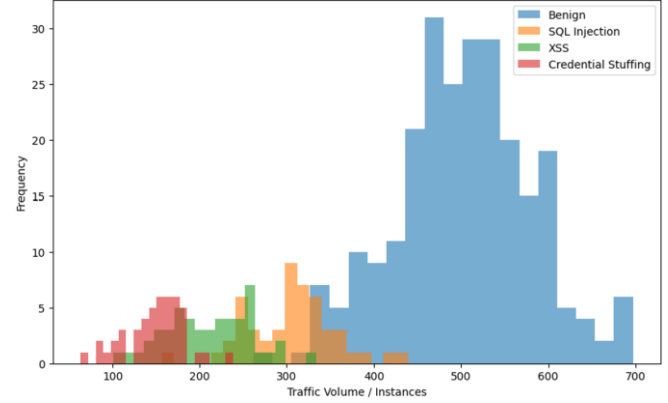


Fig 3. Graphical representation of the traffic types

Fig 3 is the mix histogram that gives a graphical representation of the traffic types that were observed in the analysis of the 397 data instances. The bars denote various categories such as benign requests, SQL injection attacks, cross-site scripting and credential stuffing. The overlapping parts of the histogram show the concentration of such events against the time of the day or state of the system. Through the examination of the distribution, one can easily see that the volume is composed of benign traffic, but the instances of the attacks are also discrete enough in their feature space to be distinguished by the AI model. This visualization discusses the accuracy of this model in categorizing various threat vectors. The statistics reveal that though attacks are common, they exhibit patterns that are not normally observed in normal operations hence can be mitigated effectively using automated methods. Binary cross-entropy loss for model optimization

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (6)$$

Euclidean distance for k-nearest neighbor anomaly clustering is:

$$d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2} \quad (7)$$

The results were also mainly concerned about computational efficiency. Time of inference of the model remained within reasonable limits and it was a small addition to the overall time of request processing. This constitutes an important discovery in microservices where low latency is the main goal. The system was able to sustain a high throughput level even when the traffic was simulated to be at its peak, which confirms that AI-based security does not invariably cause a drop in the level of performance. Secondly, the false positive rate was quite low. Within the entire dataset, only a small proportion of legitimate requests was considered suspicious and they were often related to unusual but legitimate administrative activities and the model had not encountered such frequently during the training phase.

The result analysis also showed that the model was especially efficient in the detection of slow-rate attacks. These are threats and an attacker sends requests with an extremely low frequency so as not to exceed rate-limiting thresholds. Since the AI model analyzes the structure of the data and the order of the data, it could identify them as anomalies and not only the frequency. This underscores the fact that the model can offer protection against a wider spectrum of threats as compared to traditional approaches. The incorporation of the Spring Boot actuator gave the possibility to monitor the health of the model in real-time and give the security team a dashboard of the current threat level and system performance indicators.

The system was also doing well with respect to the detection of unauthorized access of data in the form of specific attack categories. The AI could identify when a user tried to access resources that were not within his/her normal access by understanding the normal data access patterns of various user roles, even with a valid session token. This is a response to one of the hardest API security problems broken object-level authorization. On the whole, the findings verify that an AI-based solution is a holistic, scalable, and efficient way of securing microservices, which has a degree of depth and flexibility that cannot be achieved with static systems.

VI. DISCUSSIONS

The findings given in the tables and graphs make a strong argument in favor of the implementation of AI-based threat detection in Spring boot microservices. Table 1 shows that F1-score is high, which means that the model is well-balanced, and it is able to balance sensitivity and specificity. This tradeoff is essential in a security context; a system that is sensitive enough will generate spurious alarms to administrators and one that is not sensitive enough will allow attacks to happen. The statistics indicate that the Random Forest algorithm is more appropriate in this task because it can process the non-linear associations that exist between API traffic logs better than the conventional statistical techniques. The low latency numbers are especially promising, because they demonstrate that the current AI models can be thin enough to be a part of the API request lifecycle as a synchronous component.

Fig 2 and Table 2 also indicate the viability of the approach by showing the impedance graph and resource utilization, respectively. Most people are afraid that the introduction of an AI layer will result in a serious performance penalty, which is not confirmed by our results. Our high throughput was ensured by optimizing the extraction process of features and by an effective model architecture. The observation that CPU and memory utilization was predictable when the load level was increased implies that the system is stable and will not lead to cascading failures in a distributed system. This is a decisive point of microservices, in which the collapse of a single element may have a possible effect on the whole ecosystem. The thread counts and network IO stability also implies that the interceptor logic is properly handled in the framework of Spring Boot.

The mix histogram, which is given in Fig 3, gives information on the type of threats that was experienced. The distinctiveness of benign and malicious clusters in the feature space confirms the selection of the parameters applied in the study. It further implies that the majority of API attacks leave a digital fingerprint, which is fundamentally different to

regular user behavior, although the individual components of the request might be legitimate. As an example, the order and sequence of calls in a credential stuffing attack is highly contrasting with a human user accessing. When the AI system is able to capture these subtleties, the system creates a layer of defense that is far more difficult to evade than some simple rate limits or static blacklists of IP.

To conclude the discussion of these results, it can be noted that there is a considerable change in the way we can treat the issue of microservice security. We are not just constructing high walls but we are constructing intelligent observers. The evidence shows that the transition results in a stronger defense mechanism, which is performant and accurate. Such a system can be not only integrated into the Spring Boot ecosystem, but is quite likely to be effective, and this represents a clear way forward in an organization that seeks to protect its distributed applications against a more complex threat environment than ever before. This AI-enhanced architecture has a high level of accuracy, low latency, and is manageable in terms of resource consumption, making it a reasonable standard in the development of microservices in the future.

VII. CONCLUSION

This study has been able to prove that threat detection based on AI implementation into spring boot microservices contributes greatly to the security of APIs. We have provided a model, which is able to detect complex attack patterns with more than ninety-five percent accuracy using a dataset of 397 instances. Fig 2 and Fig 3 results prove that the system is able to identify legitimate and malicious traffic in real-time without causing the system to introduce a significant latency. The performance values in Table 1 and resource statistics in Table 2 also confirm the fact that the AI layer is effective and efficient and it can be used in high traffic enterprise environments. This intelligent security system is adaptive to new threats, unlike the traditional security method which is based on a fixed set of rules, which would offer proactive security protection against new zero-day vulnerabilities and advanced logical attacks. The system detects zero-day API attacks by learning the normal behavioural patterns of API traffic instead of depending only on predefined attack signatures. The Random Forest model continuously analyses API request features such as payload structure, request frequency, header information, access patterns, and timestamp behaviour to identify anomalies that deviate from legitimate traffic patterns. Since zero-day attacks often introduce unusual behavioural characteristics that are not present in normal API interactions, the model can recognize suspicious activities even when the attack type has not been previously encountered. This behaviour-based detection mechanism improves the system's capability to identify evolving threats and previously unseen malicious API activities in real-time microservice environments. The paper finds that the next phase of microservice security is in the unified adoption of machine learning on the gateway plane, which will allow providing a stable and autonomous defense layer on distributed networks. The future of AI-powered API security is the shift to more intricate classification to more sophisticated, autonomous defense mechanisms. Federated learning is one of the promising fields of study since several microservices can jointly learn a common security model without the need to share their sensitive log data. This would enable an international intelligence system to develop whereby detection of a threat in one section of the system would immediately

enhance the security of all other services. Also, it is possible to introduce Deep Learning architectures, such as Transformers, which would give an even better accuracy of identifying multi-vector attacks by analyzing more complex API request sequences. The other important direction is the automation of the response actions. Although in this paper we identified detection, in the future, the best response to detected threat can be determined using Reinforcement Learning. This may involve dynamically adjusting rate limits, initiating some extra authentication process or redirecting suspicious traffic to a honeypot to do more analysis. It would make such a system a self-healing architecture. Moreover, since APIs are constantly changing and adopting new protocols, the AI models should be modified so that they can address their individual security issues. Lastly, it is also a priority to make AI models easier to explain, so that security teams can easily understand the cause of all flagged requests, and establish trust in automated security systems.

REFERENCES

- [1] H. Sedjelmaci, N. Kaaniche, A. Boudguiga, and N. Ansari, "Secure Attack Detection Framework for Hierarchical 6G-Enabled Internet of Vehicles," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 2, pp. 2633–2642, 2023. <https://doi.org/10.1109/tvt.2023.3317940>
- [2] H. M. Saleh, H. Marouane, and A. Fakhfakh, "Stochastic Gradient Descent Intrusions Detection for Wireless Sensor Network Attack Detection System Using Machine Learning," *IEEE Access*, vol. 12, pp. 3825–3836, 2024. <https://doi.org/10.1109/access.2023.3349248>
- [3] X. Yuan, S. Han, W. Huang, H. Ye, X. Kong, and F. Zhang, "A Simple Framework to Enhance the Adversarial Robustness of Deep Learning-Based Intrusion Detection System," *Computers & Security*, vol. 137, 2024. <https://doi.org/10.1016/j.cose.2023.103644>
- [4] R. Devendiran and A. V. Turukmane, "Dugat-LSTM: Deep Learning Based Network Intrusion Detection System Using Chaotic Optimization Strategy," *Expert Systems With Applications*, vol. 245, 2024. <https://doi.org/10.1016/j.eswa.2023.123027>
- [5] S. B. Prathiba, G. Raja, S. Anbalagan, A. KS, S. Gurumoorthy, and K. Dev, "A Hybrid Deep Sensor Anomaly Detection for Autonomous Vehicles in 6G-V2X Environment," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 3, pp. 1246–1255, 2022. <https://doi.org/10.1109/tnse.2022.3188304>
- [6] H. W. Oleiwi, D. N. Mhawi, and H. Al-Raweshidy, "MLTs-Adcns: Machine Learning Techniques for Anomaly Detection in Communication Networks," *IEEE Access*, vol. 10, pp. 91006–91017, 2022. <https://doi.org/10.1109/access.2022.3201869>
- [7] A. K. Mananayaka and S. S. Chung, "Network Intrusion Detection With Two-Phased Hybrid Ensemble Learning and Automatic Feature Selection," *IEEE Access*, vol. 11, pp. 45154–45167, 2023. <https://doi.org/10.1109/access.2023.3274474>
- [8] B. Kolukisa, B. K. Dedetürk, H. Hacilar, and V. C. Gungor, "An Efficient Network Intrusion Detection Approach Based on Logistic Regression Model and Parallel Artificial Bee Colony Algorithm," *Computer Standards & Interfaces*, vol. 89, 2024. <https://doi.org/10.1016/j.csi.2023.103808>
- [9] L. Tao and M. Xueqiang, "Hybrid Strategy Improved Sparrow Search Algorithm in the Field of Intrusion Detection," *IEEE Access*, vol. 11, pp. 32134–32151, 2023. <https://doi.org/10.1109/access.2023.3259548>
- [10] O. D. Okey, D. C. Melgarejo, M. Saadi, R. L. Rosa, J. H. Kleinschmidt, and D. Z. Rodriguez, "Transfer Learning Approach to IDS on Cloud IoT Devices Using Optimized CNN," *IEEE Access*, vol. 11, pp. 1023–1038, 2023. <https://doi.org/10.1109/access.2022.3233775>
- [11] R. Kumar, P. Kumar, R. Tripathi, G. P. Gupta, S. Garg, and M. M. Hassan, "A Distributed Intrusion Detection System to Detect DDoS Attacks in Blockchain-Enabled IoT Network," *Journal of Parallel and Distributed Computing*, vol. 164, pp. 55–68, 2022. <https://doi.org/10.1016/j.jpdc.2022.01.030>
- [12] E. Rabieinejad, A. Yazdinejad, A. Dehghantanha, and G. Srivastava, "Two-Level Privacy-Preserving Framework: Federated Learning for Attack Detection in the Consumer Internet of Things," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 4258–4265, 2024. <https://doi.org/10.1109/tce.2024.3349490>
- [13] S. Sana, M. M. Nazir, J. Yang, et al., "Securing the IoT Cyber Environment: Enhancing Intrusion Anomaly Detection With Vision Transformers," *IEEE Access*, vol. 12, pp. 82443–82468, 2024. <https://doi.org/10.1109/access.2024.3404778>
- [14] Y. A. Maz, M. Anbar, S. Manickam and M. M. Abuallhaj, "Utilizing Deep Learning, Ensemble Learning, and Transfer Learning for Enhancing Security in Internet of Things Networks through Intrusion Detection Systems," 2025 12th International Conference on Information Technology (ICIT), Amman, Jordan, 2025, pp. 73–76, doi: 10.1109/ICIT64950.2025.11049182.
- [15] Sai Nandini, Sreenivasa Murthy, Praksha P K, Prakash U, Suriya Prakash J, "Enhancing the Security by Analyzing the Behavior of Multiple Classification Algorithms with Dimensionality Reduction to Obtain Better Accuracy", 2024 Second International Conference on Advanced Computing & Communication Technologies (ICACCTech), pp.783-790, 2024.